A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Vu Nguyen[1], Dung Van Bui[2,3], Nhut Tri Do[2,3], and Van Du Nguyen[4]

[1] University of Transport and Communications, Ho Chi Minh City Campus, Vietnam
nvthai@utc2.edu.vn
[2] Vietnam National University Ho Chi Minh City, Vietnam
[3] University of Information Technology, Vietnam
dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn
[4] Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam
nvdu@hcmuaf.edu.vn

Abstract Deploying machine learning (ML) based intrusion detection on resource-constrained edge devices requires balancing accuracy, latency, and operational cost. We present a split-deployment IDS architecture across two NVIDIA Jetson Orin Nano Super devices connected through Apache Kafka. The system implements a two-stage pipeline: (1) a lightweight autoencoder anomaly gate on Jetson #1 filters benign traffic, and (2) a PySpark `PipelineModel` classifier on Jetson #2 processes only suspicious flows. Central infrastructure (Kafka, PostgreSQL, InfluxDB, Grafana) runs on a host PC. The classifier is the model selected by our companion Spark ML study under a *leakage-aware* evaluation (label-leaking port features removed, feature-level duplicate flows eliminated). We evaluate three distributed modes (pipeline split, horizontal Kafka consumer scaling, and a Spark standalone cluster) while streaming CICIDS2017 network flows. We report throughput, latency, and the gate's recall–skip trade-off across the three deployment modes: decoupling the gate from the classifier raises verdict throughput from 2.6 to 62.5 flows/s ($24\times$) at a run-level p95 of $\approx$13 ms, with the gate skipping 95.8% of benign traffic.

Keywords: Intrusion detection · Edge computing · Jetson Orin Nano Super · Kafka · PySpark · Stream processing.

1 Introduction

Network intrusion detection systems (IDS) increasingly must operate closer to data sources to reduce bandwidth and response time [18]. Edge devices such as the NVIDIA Jetson Orin Nano Super offer limited CPU, memory, and thermal headroom, making monolithic ML inference challenging. A practical edge IDS must therefore decide *which* model to deploy—one that is small, accurate, and explainable—and *how* to distribute inference across nodes under streaming load.

Contributions. (1) A two-stage distributed IDS combining a lightweight anomaly gate with a Spark-based classifier; (2) Three deployment modes for a two-node Jetson

cluster (pipeline split, horizontal scaling, Spark standalone cluster); (3) An end-to-end performance evaluation reporting throughput, latency percentiles, and per-node resource usage, which are metrics that offline studies typically omit; (4) Open, reproducible scripts integrated with a *leakage-aware* Spark ML training pipeline, so the deployed model is the one validated under a rigorous offline protocol; all code and configurations are publicly available[5].

2 Related Work

Edge computing pushes analytics closer to data sources to reduce latency and bandwidth [18, 6]. Prior edge IDS studies [16, 3, 2] benchmark single-node inference on resource-constrained boards, including recent NVIDIA Jetson deployments, but they rarely evaluate multi-node Kafka pipelines with explicit role separation. Streaming platforms such as Kafka [9] decouple producers and consumers; however, most ML-based IDS papers [8, 5, 19] report offline batch accuracy without end-to-end throughput or per-node resource traces.

Split inference. Anomaly-detection surveys [4, 15] motivate two-stage designs: a lightweight gate followed by a heavier classifier. Autoencoder-based detectors [11, 7] and federated edge IDS [13] explore similar ideas on servers or homogeneous edge nodes, and recently on a single Jetson Nano [7], but none use PySpark MLlib classifiers on the NVIDIA Jetson Orin Nano Super [14]. Our companion ML study selects a leakage-free SHAP Top-30 model on CICIDS2017 [17, 12]; this paper focuses on *how* to deploy that model across two Jetsons with a measurable gate-skip ratio and three scaling modes. Table 1 positions our system against representative related work.

Table 1. Capability comparison with prior edge IDS deployments ("—": not reported or not applicable).

Work	Platform	Multi-node roles	Streaming ingest	Energy & p95 latency	Leakage-aware model	Open code
Sforzin et al. [16]	Raspberry Pi	No	No	—	—	—
López-Martín et al. [11]	GPU server	No	No	—	—	—
Nguyen et al. [13]	IoT gateways	Federated	No	—	—	—
Hasan et al. [7]	1× Jetson Nano	No	No	—	—	—
Baalaji et al. [3]	IoT-edge board	No	No	—	—	—
Asghar et al. [2]	Edge device	No	No	—	—	—
This work	**2× Jetson Orin Nano S.**	**Yes**	**Yes (Kafka)**	**Yes**	**Yes**	**Yes**

3 System Architecture

3.1 Split Deployment Overview

Figure 1 shows the split-deployment architecture. A host PC runs Kafka, PostgreSQL, InfluxDB, and Grafana. Two Jetson Orin Nano Super nodes consume network-flow

[5] `https://github.com/vuthainguyen1602/Thesis_IDS` (branch `develop`).

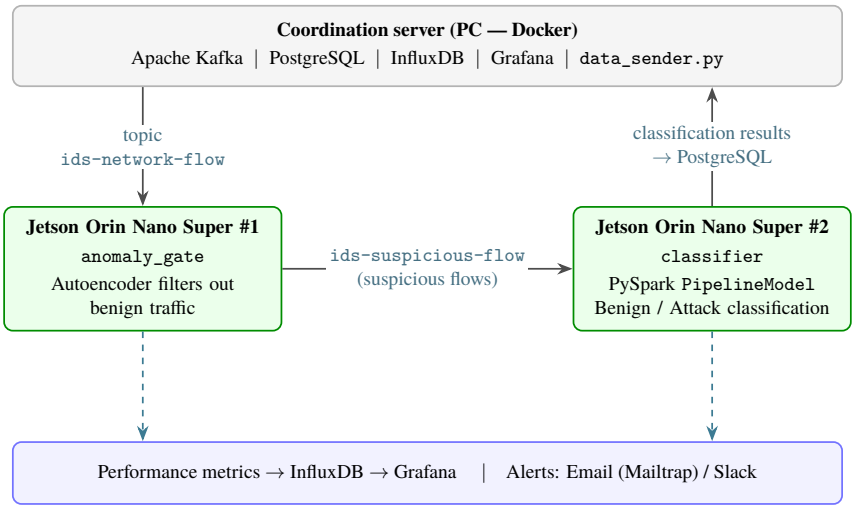

Figure 1. Split-deployment architecture (PC + 2× Jetson Orin Nano Super).

messages from topic `ids-network-flow`. In pipeline-split mode, Jetson #1 runs the anomaly gate and forwards only *suspicious* flows to topic `ids-suspicious-flow`. Jetson #2 then applies the PySpark classifier and writes the classification results back to PostgreSQL. Both nodes stream performance metrics to InfluxDB/Grafana and raise alerts via Email/Slack (dashed arrows in Fig. 1).

3.2 Pipeline Roles

- **anomaly_gate:** sklearn autoencoder + scaler; the MSE threshold is calibrated offline (Exp 8) as a quantile of a held-out *benign validation split* (10% of benign training flows, never the test set), so the operating point is committed before evaluation.
- **classifier:** PySpark `PipelineModel` with the leakage-free SHAP-selected feature set.
- **full:** combined pipeline for horizontal scaling via Kafka consumer groups.

3.3 Monitoring and Alerting

Each node tags metrics with `EDGE_NODE_ID` in InfluxDB and PostgreSQL. Alerts are emitted from a designated primary node only, avoiding duplicate notifications. Figure 2 shows the live Grafana dashboard while CICIDS2017 flows are replayed through the split deployment: prediction throughput, detected-attack count, inference latency, and per-node CPU/RAM/temperature are all tracked in real time, alongside the most recent attack alerts persisted to PostgreSQL.

Figure 2. Real-time Grafana monitoring of the running edge IDS: per-node throughput, detected attacks, inference latency, and CPU/RAM/temperature for both Jetson Orin Nano Super nodes, plus recent PostgreSQL-backed attack alerts.

4 Implementation

Training and model export run on a Spark standalone cluster (host PC as master, two Jetson Orin Nano Super devices as workers) using Apache Spark ML [20]; the cluster topology is shown in Fig. 3. The host only coordinates (Spark master) and does *not* run an executor, so all reported training compute reflects the two homogeneous Jetson workers rather than the stronger host CPU. The edge stack is implemented in Python (`jetson/edge/`) with configurable roles via environment variables. Every offline run logs its configuration (random seed, leakage/dedup switches, library versions) so each exported model is traceable to the exact pipeline that produced it.

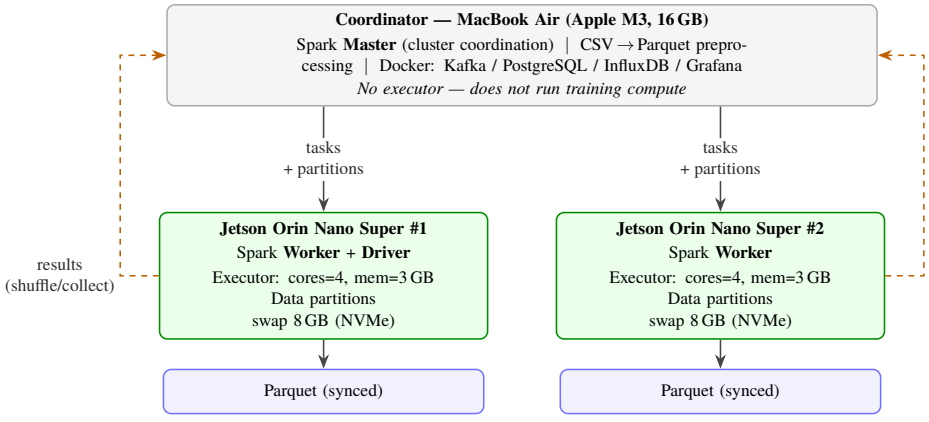

Figure 3. Apache Spark standalone training cluster: the host acts as *master* (coordination only, no executor), while two Jetson Orin Nano Super devices are the *worker/executor* nodes; Parquet data is partitioned and synced to both, with results gathered via shuffle/collect.

5 From Spark Training to Edge Deployment

This paper does *not* compare Spark against Jetson Orin Nano Super as competing platforms. The two components address complementary phases of the same train–deploy pipeline (Fig. 4). Spark answers *which model and how many features* achieve acceptable accuracy on the full CICIDS2017 dataset [17], whereas the Jetson Orin Nano Super answers *whether that model can run in real time* under memory, thermal, and bandwidth constraints at the network edge [18].

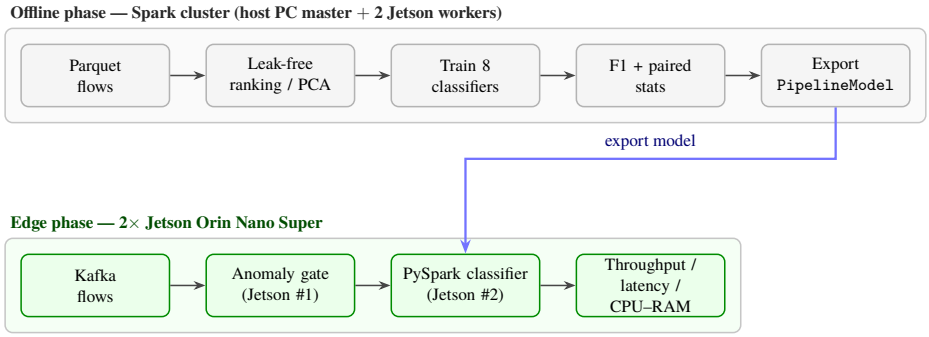

Figure 4. Two-phase train–deploy pipeline linking Spark ML experiments to Jetson edge inference.

5.1 Offline Phase (Spark ML), Leakage-Aware

In our companion ML study, we evaluate four feature configurations—baseline (all features), RF Importance Top-30, SHAP Top-30, and PCA (k=40)—under a fixed 80/20 train–test split, using eight Spark ML classifiers and statistical validation (repeated paired resampling and permutation tests). PCA uses k=40 rather than 30 because it *extracts* components (linear combinations spreading variance across many axes) rather than *selecting* original features, so it needs more components to retain comparable variance; across the common $\{20, 30, 40\}$ sweep, k=40 retains the most variance and is the fairest representative operating point for PCA. LightGBM is excluded because its SynapseML native libraries are x86_64-only and cannot run on the ARM64 Jetson Orin Nano Super deployment target. Crucially, the offline protocol is *leakage-aware* [1]: it removes information that would be unavailable at deployment time but that otherwise inflates test scores. Motivated by documented labeling and duplication errors in CICIDS2017 [10], the label-leaking `destination_port` feature is removed and flows that are identical on all feature columns are de-duplicated before splitting. This yields realistic F1 values (Table 2) rather than the near-perfect scores obtained when these leaks are left in place. Offline metrics constrain *which* exported model is worth deploying; they are not directly comparable to edge throughput or latency.

Table 2. Bridge table: offline Spark selection criteria (leakage-free Exp 7 on CICIDS2017; values from `cross_method_summary.csv`).

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003†	Explainable; fewer features (deployed feature set)
RF Top-30 + XGBoost	0.9922	30	2.52	Same feature count; embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduction

†Serialized on the training cluster, where the distributed model-saving step scatters the tree-data partitions across executors; KB-scale entries captured only driver-local metadata and under-report the true size (a 30-feature XGBoost is in reality ≈2.5 MB, on par with RF Top-30, since ensemble size depends on trees/nodes rather than input features; with fewer split candidates trees may even need slightly more nodes than the baseline, cf. 2.52 vs. 2.32 MB). The single-node export used for deployment measures sizes reliably (cf. Table 5).

5.2 Edge Phase (Jetson Orin Nano Super)

The exported `PipelineModel` (VectorAssembler, StandardScaler, classifier) is loaded on Jetson #2 with only the selected SHAP features, reducing assembly and inference cost relative to the 77-feature baseline. The deployed input vector contains 29 features: the SHAP Top-30 ranking includes `destination_port`, which is excluded from the exported model as a label-leaking channel on CICIDS2017, consistently with the leakage-aware offline protocol. A lightweight sklearn autoencoder gate (Exp 8) on Jetson #1 filters benign flows before they reach Spark inference, further lowering load on the classifier node. End-to-end edge evaluation measures throughput (flows/s), latency p50/p95 (ms), per-node CPU/RAM/temperature, attack-class F1, and gate-skip ratio, none of which the offline Spark experiments capture.

5.3 Division of Evaluation Metrics

Table 3 clarifies which metric belongs to which phase. Mixing offline F1 with edge latency in a single ranking would conflate accuracy with operational feasibility.

Table 3. Metric split between offline Spark training and edge Jetson inference.

Aspect	Spark (offline)	Jetson (edge)
Primary goal	Classification accuracy	Real-time feasibility
Main metrics	F1, AUC-PR, Precision/Recall	Throughput, latency p50/p95
Secondary metrics	Train time, batch pred. time, model size	CPU, RAM, temperature, gate skip %
Data	CICIDS2017 train/test split	Kafka replay at 100 flows/s
Scale	Millions of flow records	2 nodes, 8 GB RAM each

We deploy the SHAP Top-30 feature set on Jetson #2 with a Random Forest classifier (offline F1 0.9955, within 0.002 of the best XGBoost score in Table 2): it retains near-baseline F1 with substantially fewer features, and Random Forest is a native Spark ML estimator, so the exported `PipelineModel` loads on the ARM board without the additional native libraries that Spark's XGBoost integration requires. Section 7 then evaluates this choice under streaming load; the companion ML paper reports the full four-method comparison and drift analysis.

6 Experimental Setup

6.1 Hardware and Software

2× NVIDIA Jetson Orin Nano Super Developer Kit (6-core Arm Cortex-A78AE, NVIDIA Ampere GPU with 1024 CUDA + 32 Tensor cores, 8 GB LPDDR5, 256 GB NVMe SSD, ∼67 TOPS), host PC running Docker Compose, Wi-Fi LAN. PySpark 3.4.1, Apache Kafka 3.5 (Confluent Platform 7.5.0), InfluxDB and Grafana for metrics.

6.2 Workloads

CSV replay through `data_sender.py` at 100 flows/s over CICIDS2017. Reported metrics: throughput, p50/p95 latency (per-node percentiles over raw per-flow samples; the cluster p95 is the worst node's p95, plus end-to-end send-to-verdict latency from producer timestamps under NTP-synchronized clocks), per-node CPU/RAM/temperature, attack-class F1, gate-skip ratio, and energy-per-inference (mJ, via `tegrastats`, idle baseline subtracted).

6.3 Baselines

Single-node full pipeline vs. distributed modes A (pipeline split), B (horizontal Kafka scaling), and C (Spark standalone cluster).

7 Results

We run each deployment mode through a host-side orchestrator (`run_dist_bench.sh`) that starts the corresponding pipeline roles on the Jetson nodes over SSH, drives the load, and collects per-node logs. Each configuration is measured over at least three repetitions preceded by representative warmup traffic (excluded from the measured window, so JVM/JIT warmup does not inflate early-batch latency); Table 4 reports mean ± std across repetitions. All metric queries are aligned to the exact load window rather than a trailing time range. Latency percentiles are computed per node over the raw per-flow samples logged during the window, and pipeline throughput is counted as *final verdicts per second* (a flow's verdict is issued either at the gate, for skipped flows, or at the classifier), which avoids double-counting forwarded flows in pipeline-split mode. Aggregated results are reported in Table 4, with throughput and p95 latency across modes visualized in Fig. 6.

Table 4. Distributed deployment comparison on the 2-node Jetson cluster (CICIDS2017 replay at 100 flows/s; mean ± std over ≥3 repetitions; p95 from raw per-flow logs, worst node). "(inline)": in the `full` role the gate runs in-process and its skips are persisted as verdicts, so no separate skip ratio applies. Energy is *total-board* energy per verdict: at this load the idle-subtracted (active) power delta was below the `tegrastats` noise floor (<0.15 W), so the figure is dominated by board idle power and measures how effectively each mode amortizes it over verdicts. † Detection quality is mode-independent (identical exported models); offline attack-class F1 is reported in Table 2.

Mode	Throughput (flows/s)	Latency p95 (ms)	Gate skip (%)	Energy/inf. (mJ)	Attack F1
Single node	2.60 ± 0.41	12.3 ± 3.0	(inline)	2490	†
A: Pipeline split	62.5 ± 0.6	13.1 ± 2.6	95.8	178	†
B: Horizontal	5.9 ± 1.4	21.6 ± 19.2	(inline)	2250	†
C: Spark cluster	90.0 ± 25.6	23.9 ± 17.8	97.9	119	†

Energy efficiency. On-board power is sampled with `tegrastats` during each run (30 s idle baseline, then the load window; for two-node modes the powers of both nodes are summed). At 100 flows/s the measured *active* (idle-subtracted) power delta was below the sampling noise floor (<0.15 W on every node), so Table 4 reports *total-board* energy per verdict instead: essentially board power amortized over verdict throughput. This still separates the modes sharply — the gate-based modes (A and C) amortize the two boards' power over 24–35× more verdicts than the single node, cutting energy per verdict from ≈2.5 J to 0.12–0.18 J — but the figures should be read as a deployment-level (not model-level) energy metric.

Figure 5 shows the PySpark classifier node (`jetson-nano-2`) processing replayed CICIDS2017 traffic in operation: it sustains a steady final-verdict throughput (≈6 flows/s in this window), per-batch inference latency of a few milliseconds, and correct per-batch attack counts, while the per-node monitor reports live CPU/RAM/temperature.

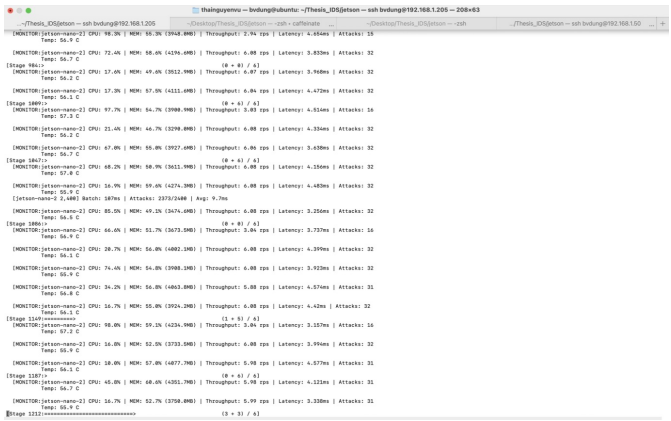

Figure 5. Classifier node (`jetson-nano-2`) in operation: per-batch attack detection, few-millisecond inference latency, and live per-node resource telemetry during CICIDS2017 replay.

7.1 Inference-engine baseline (cost of the unified stack)

We deploy a Spark `PipelineModel` on the edge for *consistency with the distributed training phase* — the same stack trains on the cluster and serves on the device. To quantify the price of that unification, we benchmark the *same* RandomForest (matched hyper-parameters) on the *same* deployed SHAP-selected feature set (Sec. 5) through lighter single-node engines: scikit-learn (CPU, no JVM) and ONNX Runtime (Table 5). Accuracy is comparable by construction; the comparison targets inference cost. As Table 5 shows, the JVM/Spark stack indeed incurs substantially higher inference cost than scikit-learn on the 8 GB ARM board (7.7× lower throughput, ≈8× higher p95, double the RAM share); reporting this honestly motivates exporting the `PipelineModel` to ONNX/TensorRT for inference while retaining Spark for training, which we leave as future work.

Table 5. Single-node inference cost on the same model/features (from `benchmark_engines.py` vs `benchmark.py`).

Engine	Throughput (flows/s)	p95 latency (ms)	RAM (%)	Energy/inf. (mJ)
PySpark (deployed)	26.8	397	27.5	–
scikit-learn	205.5	49.8	12.7	2.87
ONNX Runtime	not measured (onnxruntime not installed on ARM64)			

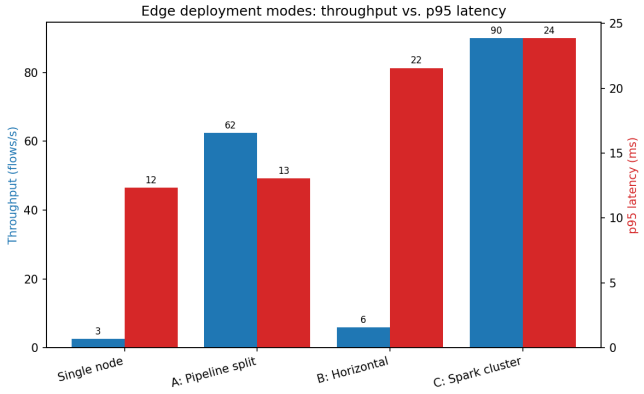

Figure 6. Throughput and p95 latency across deployment modes (from `summary.csv`).

The anomaly gate is a *tunable* filter rather than a single point: sweeping its threshold trades attack recall against the fraction of benign flows offloaded from Spark (Fig. 7). This lets an operator pick an operating point that fits the available edge budget.

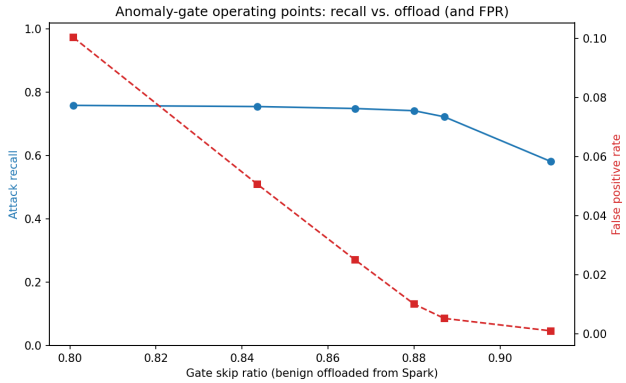

Figure 7. Anomaly-gate operating curve: attack recall vs. gate-skip ratio (and FPR).

8 Discussion

Pipeline split relieves Spark of most benign traffic, and the realized benefit scales with the gate-skip ratio ($\approx$96% in our runs): decoupling the gate from the Spark classifier lifted verdict throughput from 2.6 to 62.5 flows/s ($24\times$) at essentially unchanged p95. Horizontal scaling of the *full* role roughly doubles the single node (5.9 vs. 2.6 flows/s) but remains Spark-bound, because every node still runs the classifier in-process. The Spark standalone cluster variant reached the highest mean throughput (90 flows/s) but with large run-to-run variance ($\pm$26) and an extra host dependency (the Spark master), so pipeline split remains the recommended default; cluster mode is most useful when model or batch size exceeds single-node memory.

As summarized in Sec. 5, offline Spark experiments and edge Jetson benchmarks answer different questions. Feature reduction from 77 to 30 dimensions (SHAP Top-30) directly reduces `VectorAssembler` and `PipelineModel` cost on the Orin Nano Super, while the anomaly gate skips a large fraction of benign flows before Spark inference is invoked. Together, these design choices connect the accuracy-oriented conclusions of the companion ML study to a deployable, measurable edge architecture.

8.1 Limitations

Benchmarks are obtained on a two-node lab cluster over Wi-Fi; wired networking and larger clusters may shift latency tails. The anomaly-gate threshold trades attack recall against the gate-skip ratio, so we report it as an operating curve (Fig. 7) and leave live-traffic re-calibration to future work. Each configuration is repeated at least three times with warmup excluded (Sec. 7); run-to-run variance on a thermally constrained board remains non-trivial and is reported alongside the means. Latency percentiles are computed per node from raw per-flow samples, and the cluster-level p95 is conservatively the *worst node's* p95; end-to-end latency (from producer send to final verdict) relies on NTP-synchronized clocks and excludes DB-write cost. Running PySpark on the JVM on an 8 GB Jetson incurs substantial overhead: Spark is chosen for consistency with the distributed training phase, the overhead is quantified in Sec. 7.1, and the $\sim$67-TOPS

GPU remains unexploited by the CPU-only pipeline (TensorRT is future work). The energy-per-inference figures subtract a single pre-run idle baseline, so within-run drift in background power is not captured; in split mode they sum both nodes' active energy, and the gate-skip saving does not subtract the autoencoder cost incurred on all flows. Finally, results are tied to CICIDS2017 replay: live traffic and deliberately evasive (adversarial) traffic remain untested.

9 Conclusion

We presented a reproducible distributed edge IDS for Jetson Orin Nano Super clusters with Kafka streaming and PySpark inference, deploying a model validated under a leakage-aware offline protocol. Future work includes TensorRT optimization, live-traffic validation, and federated retraining under drift.

Acknowledgments. The work was carried out at the University of Transport and Communications (Ho Chi Minh City Campus) and the University of Information Technology, VNU-HCM, as part of a graduation thesis research project.

Disclosure of Interests. The authors declare that they have no competing interests.

References

1. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
2. Asghar, M.Z., et al.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. Scientific Reports **16**, 2313 (2026). https://doi.org/10.1038/s41598-025-32183-8
3. Baalaji, K., et al.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. Discover Applied Sciences **7**, 928 (2025). https://doi.org/10.1007/s42452-025-06871-z
4. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Computing Surveys **41**(3), 1–58 (2009). https://doi.org/10.1145/1541880.1541882
5. Ferrag, M.A., Maglaras, L., Janicke, H., Jiang, J., Shu, L.: Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. Journal of Information Security and Applications **50**, 102419 (2020)
6. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. Sensors **20**(22), 6441 (2020). https://doi.org/10.3390/s20226441
7. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), arXiv:2501.15266
8. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J.: Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity **2**(1), 1–22 (2019)
9. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (2011)

10. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In: Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers. pp. 18–33. Lecture Notes in Computer Science, Springer (2023). https://doi.org/10.1007/978-3-031-31108-6_2
11. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in iot. Sensors **17**(9), 1967 (2017). https://doi.org/10.3390/s17091967
12. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems **30** (2017)
13. Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.R.: DÏoT: A federated self-learning anomaly detection system for IoT. In: Proc. IEEE 39th Int. Conf. on Distributed Computing Systems (ICDCS). pp. 756–767 (2019). https://doi.org/10.1109/ICDCS.2019.00080
14. NVIDIA Corporation: Jetson orin nano super developer kit (2024), `https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/`
15. Ruff, L., Kauffmann, J.R., Vandermeulen, R.A., Montavon, G., Samek, W., Kloft, M., Dietterich, T.G., Müller, K.R.: A unifying review of deep and shallow anomaly detection. Proceedings of the IEEE **109**(5), 756–795 (2021)
16. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom). pp. 440–448 (2016). https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCom-IoP-SmartWorld.2016.0080
17. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116 (2018)
18. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet of Things Journal **3**(5), 637–646 (2016). https://doi.org/10.1109/JIOT.2016.2579198
19. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep learning approach for intelligent intrusion detection system. IEEE Access **7**, 41525–41550 (2019)
20. Zaharia, M., Chowdhury, M., Franklin, M.J., Shenker, S., Stoica, I.: Spark: Cluster computing with working sets. In: Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10) (2010)